



Vikings War Game

Vikings - Mini Project

Group:
Diogo Silva
Rafael Rocha
Hayley Lee
Frank Sarfo
Bosco

Introduction

Mini-Project: Vikings vs. Saxons

- **Topic:** Inheritance and OOP (Object-Oriented Programming) in Python
- **Goal:** Modify `vikingsClasses.py` until **all tests pass**
- **Test files:** `1-testSoldier.py`, `2-testVikings.py`, `3-testSaxons.py`, `4-testWar.py`



Game Story

- Two rival armies: **Vikings** and **Saxons**
- Each warrior is a **Soldier** with **health** and **strength**
- The war unfolds in rounds — random attacks until one army wins

Architecture (OOP Design)

Classes & Inheritance

- `Soldier` (base class): defines `health`, `strength`
- `Viking(Soldier)`: adds `name` and custom damage messages
- `Saxon(Soldier)`: no name; unique damage messages
- `War`: orchestrates both armies and manages battles

Core Concepts Used

- **Encapsulation** (attributes & methods per class)
- **Inheritance** (Viking & Saxon reuse Soldier logic)
- **Polymorphism** (different `receiveDamage` behaviors)

Class Soldier

```
# Soldier # Parent class
class Soldier:

    def __init__(self, health, strength): # The constructor method initializes each new Soldier object
        # it takes two arguments => health and strength
        self.health = health #The health attribute shows how much life the soldier has
        self.strength = strength #The strength attribute shows how powerful the soldier's attack is

    def attack(self): # The attack() method to simulate the soldier attacking an enemy
        return self.strength # It returns the soldier's strength value as the damage received

    def receiveDamage(self, damage): # The receiveDamage() method to simulate the soldier taking damage from an attack
        self.health -= damage # The soldier's health subtracted from the damage

#Example
soldier1=Soldier(100, 50) #creating soldier 1 with 100 health and 50 strength
print(soldier1.attack()) #return the soldier's strength : output is 50
print(soldier1.receiveDamage(20)) #soldier taking 20 damage
print(soldier1.health) #shows health reduced after damage: out is 80
```

Class Viking

```
# Viking #Child Class
class Viking(Soldier):
    def __init__(self, name, health, strength): #The constructor method initializes each Viking object
                                                #it takes 3 arguments => name, health, strength
        super().__init__(health, strength) #Using super() to call the parent class Soldier constructor
        self.name = name #Viking class has its own unique attribute:name

    def battleCry(self): #The battleCry() method returns a specific war cry to the Viking
        return "Odin Owns You All!"

    def receiveDamage(self, damage): #Overriding the method from Soldier but respond differently when taking damage
        self.health -= damage
        if self.health > 0:
            return f"{self.name} has received {damage} points of damage"
        else:
            return f"{self.name} has died in act of combat"

#Example
viking1=Viking("Rafael", 150, 90) #creating a new viking named Rafael with 150 health and 90 strength
viking2=Viking("Hayley", 120, 70) #creating a new viking named Hayley with 120 health and 70 strength

print(viking1.battleCry()) #"Odin Owns You All!"
print(viking2.attack()) #70 (inherited from Soldier)
print(viking1.receiveDamage(40)) #Rafael has received 40 points of damage
```

Class Saxon

```
# Saxon #Child Class
class Saxon(Soldier):
    def __init__(self, health, strength):    #The constructor method initializes each Saxon object
                                            #it takes 2 arguments => health, strength
        super().__init__(health, strength)  #Using super() to call the parent class Soldier constructor

    def receiveDamage(self, damage): #Overriding the method from Soldier, and it defines how saxon reacts when taking damage
        self.health -= damage
        if self.health > 0:
            return f"A Saxon has received {damage} points of damage"
        else:
            return f"A Saxon has died in act of combat"

#Example
saxon1=Saxon(60, 30) #creating a new saxon with 60 health and 30 strength
saxon2=Saxon(100, 60) #creating a new saxon with 100 health and 60 strength
print(saxon1.attack()) # 30 (inherited from Soldier)
print(saxon2.receiveDamage(60)) # Saxon2 has died in act of combat
```

Wargame - Generating Vikings and Saxon Attributes

```
import random
import time      # Import time module for adding delays between actions

def generate_viking():      # Function to create random Viking characters
    names = ["Ragnar", "Lagertha", "Bjorn", "Ivar", "Floki", "Harald", "Erik"]
    name = random.choice(names) # Pick random name from list
    health = random.randint(80, 100) # Random health between 80-100
    strength = random.randint(40, 70) # Random strength between 40-70
    return Viking(name, health, strength) # Create and return Viking object

def generate_saxon():      # Function to create random Saxon characters
    health = random.randint(50, 70) # Random health between 50-70
    strength = random.randint(30, 50) # Random strength between 30-50
    return Saxon(health, strength) # Create and return Saxon object
```

Wargame - Defining the Battle

```
def start_battle():    # Main function that runs the entire battle game

    war = War()        # Create a new War object to manage the battle

    viking_count = random.randint(2, 3)    # Random number of Vikings (2 or 3)
    saxon_count = random.randint(2, 4)     # Random number of Saxons (2, 3, or 4)

    for _ in range(viking_count):          # Create all Viking warriors
        war.addViking(generate_viking())   # Add random Viking to army
    for _ in range(saxon_count):           # Create all Saxon warriors
        war.addSaxon(generate_saxon())     # Add random Saxon to army

    print("BATTLE BEGIN!")
    print("Vikings vs Saxons!")
    print()

    print("VIKING ARMY:")                  # Display Viking army stats
    for viking in war.vikingArmy:          # Loop through each Viking
        print(f" {viking.name} - Health: {viking.health}, Strength: {viking.strength}") # Show Viking info

    print("SAXON ARMY:")                  # Display Saxon army stats
    for i, saxon in enumerate(war.saxonArmy, 1): # Loop through Saxons with numbers
        print(f" Saxon {i} - Health: {saxon.health}, Strength: {saxon.strength}") # Show Saxon info
    print()

    round_number = 1                      # Start counting rounds from 1
```

Wargame - Defining the Battle / Showing the Result

```
# Loop until someone wins
while war.saxonArmy and war.vikingArmy: # Continue while both armies have warriors
    print(f"--- Round {round_number} ---") # Show current round number

    # Viking attacks
    if war.vikingArmy and war.saxonArmy: # Check if both armies still exist
        result = war.vikingAttack() # Execute Viking attack
        print(f"Viking Attack: {result}") # Show attack result
        time.sleep(1) # Pause for 1 second for better readability

    # Saxon attacks
    if war.saxonArmy and war.vikingArmy: # Check if both armies still exist
        result = war.saxonAttack() # Execute Saxon attack
        print(f"Saxon Attack: {result}") # Show attack result
        time.sleep(1) # Pause for 1 second for better readability

    print(f"Status: Vikings [{len(war.vikingArmy)}] vs Saxons [{len(war.saxonArmy)}]") # Show remaining warriors
    print()

    # Wait for user to press Enter before next round
    if war.saxonArmy and war.vikingArmy: # Only wait if battle continues
        input("Press ENTER for next round...") # Pause until user presses Enter
        print()

    round_number += 1 # Increase round counter for next round

# Final result
print(war.showStatus()) # Display who won the battle
print("BATTLE FINISHED!") # End of game message

# Execute the battle
if __name__ == "__main__": # Check if this file is being run directly
    start_battle() # Start the battle game
```

Wargame - Game Running!!!

```
D4 — python vikingsClassescopia.py — 80x24
[(base) mac@MacBook-Pro-de-mac d4 % python vikingsClassescopia.py]
BATTLE BEGIN!
Vikings vs Saxons!

VIKING ARMY:
  Floki — Health: 92, Strength: 57
  Ragnar — Health: 87, Strength: 68
SAXON ARMY:
  Saxon 1 — Health: 58, Strength: 48
  Saxon 2 — Health: 69, Strength: 44

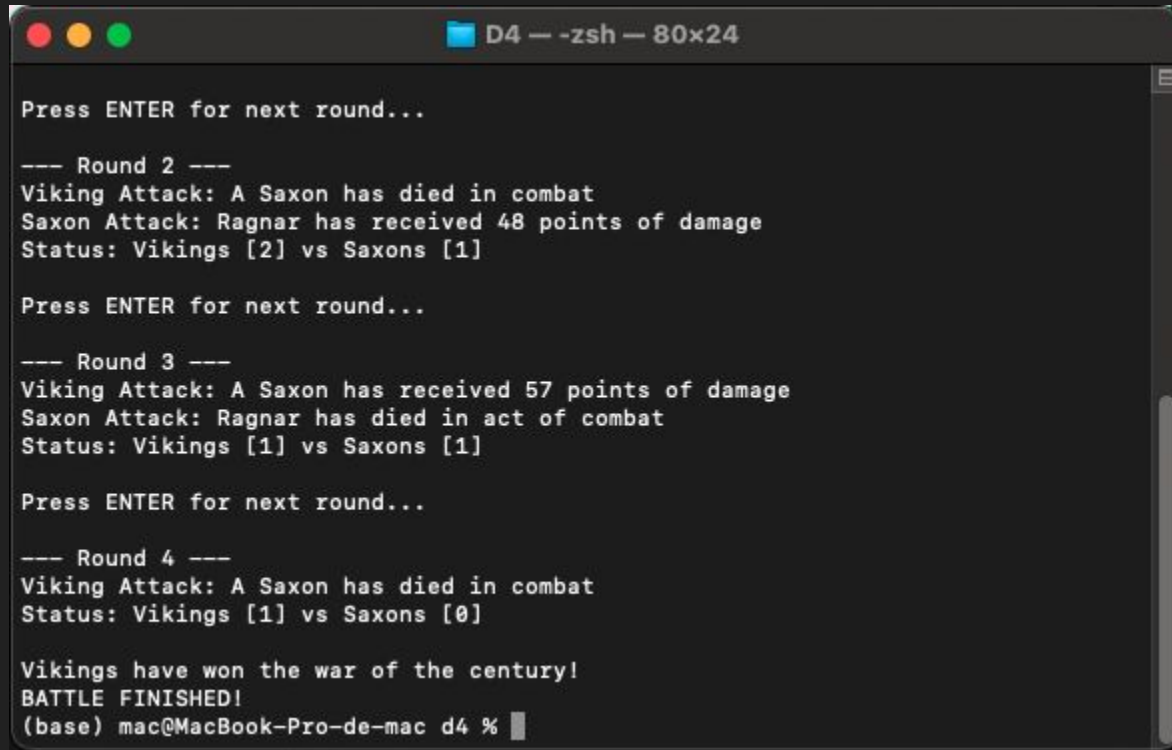
--- Round 1 ---
Viking Attack: A Saxon has received 57 points of damage
Saxon Attack: Floki has received 44 points of damage
Status: Vikings [2] vs Saxons [2]

Press ENTER for next round...

--- Round 2 ---
Viking Attack: A Saxon has died in combat
Saxon Attack: Ragnar has received 48 points of damage
Status: Vikings [2] vs Saxons [1]

Press ENTER for next round...█
```

Wargame - Up the Vikings!!!!



```
D4 --zsh-- 80x24

Press ENTER for next round...

--- Round 2 ---
Viking Attack: A Saxon has died in combat
Saxon Attack: Ragnar has received 48 points of damage
Status: Vikings [2] vs Saxons [1]

Press ENTER for next round...

--- Round 3 ---
Viking Attack: A Saxon has received 57 points of damage
Saxon Attack: Ragnar has died in act of combat
Status: Vikings [1] vs Saxons [1]

Press ENTER for next round...

--- Round 4 ---
Viking Attack: A Saxon has died in combat
Status: Vikings [1] vs Saxons [0]

Vikings have won the war of the century!
BATTLE FINISHED!
(base) mac@MacBook-Pro-de-mac d4 %
```